WholeDoc Bench: Continuity, Fidelity and Machine-Readable Content in Document Parsing

Denis Moldovan* Shrevin Sookun* Nicolae Mogage* Matus Pizur* Costinel Mitrea*
Olivier Houssin* Etienne Alby*† Coralie Carlier* Daniel Martín‡

Nutrient

{denis.moldovan, shrevin.sookun, nicolae.mogage, matus.pizur, costinel.mitrea
olivier.houssin, etienne.alby, coralie.carlier, dmartin}@nutrient.io

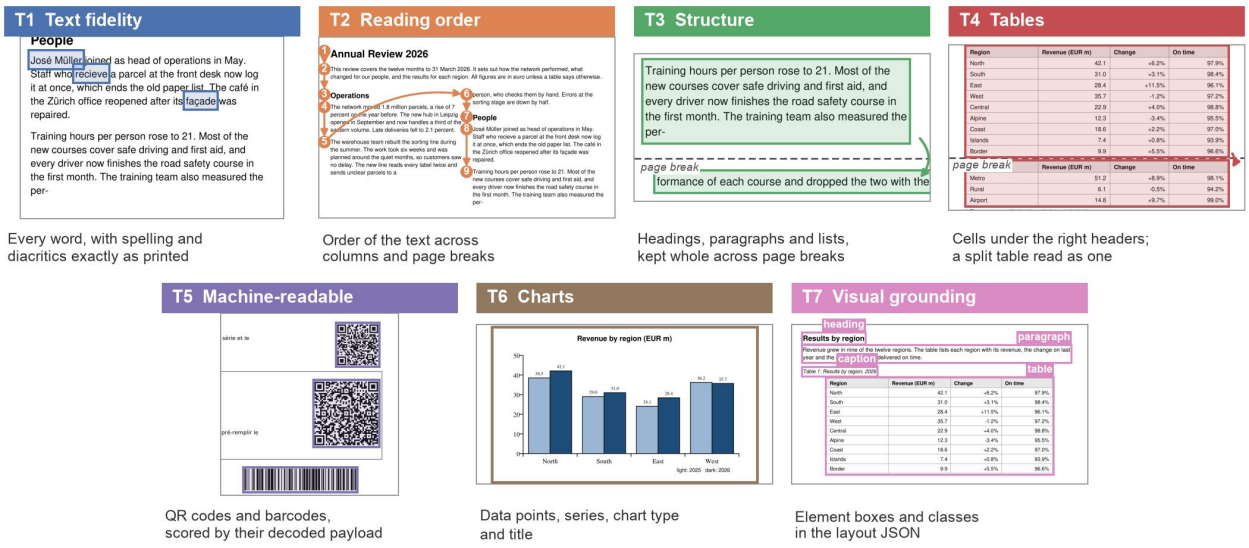


Figure 1: Examples of the ground truth read by each track of WholeDoc Bench, drawn on document pages. Several tracks follow content across page and column breaks (T2–T4), and others check content that a parser may silently change or drop, such as misspelled names (T1) and barcode and MRZ payloads (T5).

Abstract

Document parsers convert PDF files and scans into the text, tables and layout that search engines, retrieval-augmented generation pipelines, and AI agents rely on, yet most benchmarks evaluate them one page at a time. We introduce WholeDoc Bench, a benchmark that evaluates parsers on whole documents across seven tracks: text fidelity, reading order, structure and formatting, tables, machine-readable content, charts, and visual grounding. Its ground truth links paragraphs, list items, and tables split across page and column breaks, marks neighboring elements that look continuous but must stay separate, plants typos and near-miss names that must be returned as printed, and stores the decoded content of barcodes and machine-readable zones. The corpus of 481 documents is kept as a proprietary private set, and the evaluation code is released with a public sample of 10 documents. We evaluate 13 parsers in 19 configurations, including commercial document APIs, general-purpose vision-language models, and open-source tools, together with their cost per page. No parser leads every track. Vision-language models come close to specialized parsers on text, structure, and tables, but they decode almost no barcodes, give less accurate boxes, and are the most expensive configurations, and a higher reasoning effort does not improve their scores.

*Equal contribution. †Project lead. ‡Corresponding author: dmartin@nutrient.io.

1 Introduction

Documents are the main way companies, governments, and individuals store and exchange information. Contracts, financial reports, invoices, or construction documents are mostly shared as PDF files or scanned images, which are easy for humans to read, but hard for machines to process. Document parsing is the task of converting such files into a structured, machine-readable form (e.g., Markdown or JSON), which keeps the text, the reading order, the tables, the charts, and the layout of the original pages. Nowadays, the parsed output is used by search engines, retrieval-augmented generation (RAG) pipelines, and AI agents, which never see the original page like humans do, but rely only on what the parser returned [14, 36, 37].

Parsing errors can propagate downstream. A table split across two pages may be returned as two unrelated tables, and the second part loses the column headers that explain its values. A paragraph interrupted by a page break may be returned as two paragraphs, which cuts a sentence in half. Additionally, a parser may change the printed text itself, for example by replacing a misspelled name or an unusual diacritic with a more common form [11]. In all these cases the output still looks like ordinary text, even though the content of the document is no longer the same.

Several benchmarks exist for evaluating document parsers. Subtask benchmarks such as PubTabNet [38] and DocLayNet [29] focus on one capability, while end-to-end benchmarks such as OmniDocBench [28], olmOCR-Bench [30], ParseBench [36] and RealDocBench [13] cover several capabilities at once, mostly at page level. More recently, READoc [15], MPDocBench-Parse [39] and Dr. DocBench [35] extended the evaluation to whole documents or runs of pages, including paragraphs and tables truncated by page or layout boundaries. However, two aspects are still covered only indirectly. Firstly, these benchmarks reward a correct merge, but none of them contains explicit cases in which two neighboring elements look continuous and must stay separate, so a parser that merges too eagerly is penalized only through a lower similarity score. Secondly, the capabilities are spread across benchmarks: document-level continuity is evaluated in one, while chart data, text styles, and visual grounding are evaluated in others, mostly at page level, and machine-readable content such as barcodes is not evaluated in any of the benchmarks we reviewed.

We introduce WholeDoc Bench, a benchmark for evaluating document parsers on whole documents. The corpus contains 481 documents and 634 pages, annotated for content, reading order, and document structure. The benchmark is designed to test whether parsers preserve genuine continuations, keep independent content separate, and reproduce unusual printed text faithfully. Its evaluation covers seven tracks: text fidelity, reading order, structure and formatting, tables, machine-readable content, charts, and visual grounding. Fig. 1 shows one example per track, and Table 2 summarizes the evaluation metrics. The evaluation code is released together with a public sample of 10 documents and their ground truth[1][2] so that any parser can be evaluated locally, while the other documents form a proprietary private set (Section 3.5). The reported scores are computed on the full corpus.

We evaluate 13 parsers in 19 configurations through the same harness: commercial APIs (i.e., Nutrient in three modes, AWS Textract, Azure AI Document Intelligence, LlamaParse [18], Reducto [32], Extend [8] and Databricks [5]), general-purpose vision-language models (Anthropic, OpenAI), and open-source tools (Docling [7], PyMuPDF4LLM [4], pdftotext [34], MarkItDown [20]).

Our contributions are as follows:

- **Separation and continuation across breaks:** explicit pairs of neighboring elements that look continuous but must stay separate, together with links for paragraphs, list items, and tables split across page and column breaks;
- **Annotated fidelity cases:** the ground truth marks typos, diacritics, and near-miss names that must be returned as printed;
- **Machine-readable content:** the decoded payload of every barcode and QR code, and the lines and check-digit verdict of every machine-readable zone (MRZ);
- **Open evaluation on a private corpus:** the evaluation code and a public sample with ground truth are released for local evaluation, while the reported scores come from a proprietary private set that cannot be tuned against;
- **An evaluation of 13 parsers in 19 configurations,** together with their cost per page.

The paper is structured as follows. Section 2 reviews existing benchmarks for document parsing and the main families of parsing systems. Section 3 describes how documents were collected and annotated, and Section 4 defines the output format, outlines the seven evaluation tracks, and explains how their scores are aggregated. Section 5 presents the evaluated parsers, their results, and their costs, while Section 6 concludes the paper with directions for future work.

[1] https://github.com/PSPDFKit-labs/wholedoc-bench

[2] https://huggingface.co/datasets/nutrientdocs/wholedoc-bench-sample

2 Related Work

This section reviews the benchmarks proposed in the literature for evaluating document parsers, first at page level and then at document level, followed by the main families of document parsing systems.

2.1 Page-level Parsing Benchmarks

Early benchmarks target a single capability. PubTabNet [38] evaluates table structure recognition on scientific articles, using the TEDS metric, while DocLayNet [29] evaluates layout detection on 80K pages, but not the extracted content. More recent benchmarks evaluate several capabilities on the same pages. OmniDocBench [28] scores text, tables, formulas, and reading order on pages from nine document types, mostly with similarity metrics such as the normalized edit distance. olmOCR-Bench [30] replaces similarity with 7,010 binary unit tests, but on single pages and mostly academic content. ParseBench [36] extends rule-based tests to charts, semantic formatting, and visual grounding on 2K enterprise pages, and RealDocBench [13] scores the values extracted for field-level questions. All in all, these benchmarks cover a wide range of capabilities, but each evaluated sample is a single page, so content that continues on the next page is outside their scope.

2.2 Document-level Parsing Benchmarks

READoc [15] converts entire PDF files into Markdown and scores headings, formulas, tables, and plain text over the whole document. FinDocBench, introduced by Qian et al. [31] evaluates tables that span several pages by joining their fragments before computing TEDS. MPDocBench-Parse [39] also scores the merging of truncated tables, as well as paragraphs, reading order, and heading hierarchy, and Dr. DocBench [35] selects difficult pages from long books, including layouts that cross page boundaries. These benchmarks focus on content that should be merged, while cases in which two elements on consecutive pages look continuous but should stay separate receive less attention. Moreover, chart data, text styles and barcode payloads are not scored over whole documents in any of them. This is why the present approach evaluates all these capabilities on the same whole documents.

2.3 Document Parsing Systems

Pipeline-based systems, such as Docling [7] and commercial services like AWS Textract and Azure AI Document Intelligence, split parsing into layout analysis, OCR, table recognition and reading order estimation, then return the layout of each page. Lighter tools, such as pdftotext [34], PyMuPDF4LLM [4] and MarkItDown [20], read the text layer of born-digital files directly. End-to-end systems instead use a vision-language model that reads the page image, either specialized for documents, such as olmOCR [30], or general-purpose, such as Claude and GPT. These general-purpose models are billed per token and need several seconds for each page, so their cost and latency grow with the length of the document and limit their use on large collections. For this reason, ParseBench [36] and RealDocBench [13] report the cost of each system next to its accuracy, the latter also measuring the latency per page.

3 Benchmark Design and Dataset

In this section, we describe how we built the benchmark, from document collection and ground truth annotation to the annotation process and its quality control.

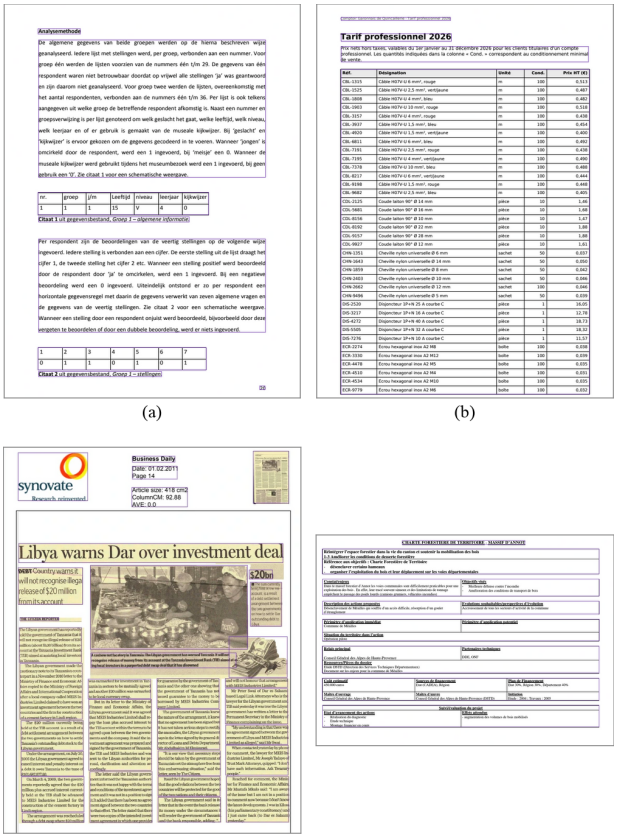
3.1 Document Collection

WholeDoc Bench contains PDF and XLSX files (Fig. 2). Other Office formats (e.g., DOCX) are planned for a future iteration of the benchmark. A benchmark document is either a whole file or a run of consecutive pages taken from a longer file, and a run never starts or ends in the middle of content that continues across a page break.

The documents come from three sources:

- **Tagged PDF files:** a pool of approximately 22,000 accessible (PDF/UA) documents with 322,000 pages, whose tag structure gives an initial version of the layout;
- **Public document collections:** publicly available documents (e.g., reports, papers, presentations, brochures), from which pages with tables, charts and multi-column layouts were selected;
- **Synthetic documents:** invoices, price lists, regulations, reports, bank statements, and export invoices, generated together with their exact ground truth in up to five languages, and rendered as born-digital or scanned files.

Candidate documents were picked in batches for variety and then reviewed by hand to keep those that are difficult for parsers. The real documents come from public sources. Before a document is sent to a parser or released in the public sample, we sanitize it: we copy its pages into a new, empty PDF file, flatten its form fields and annotations, and remove its metadata. The sanitized file renders the same pages, so its scores do not change.



(a) (b) (c) (d)

Figure 2: Sample pages of WholeDoc Bench, with the element boxes of their ground truth: (a) a report page with text and small tables, (b) a price list with a long table, (c) a scanned newspaper clipping with multiple columns, and (d) a form laid out as a grid of boxes.

3.2 Annotation Schema

Each document has one ground truth file, which lists the elements of every page, their reading order, and the relations between them. An element is a block on the page, with a box given as (x, y, w, h) in fractions of the page size, its printed text, and one of 13 classes: heading, paragraph, list item, caption, footnote, table, chart, figure, formula, code, header, footer and machine_readable. Each element also belongs to a channel, which tells the scorer how to treat it: *body* content is scored, *furniture* (i.e., running headers, footers and page numbers) is left out of the reading order, while *annotation*, *metadata* and *hidden* content costs nothing if it is returned.

Depending on its class, an element carries additional data. A heading or list item has a level. A table has its grid of header rows and body rows, in which a merged cell is written once and described by its span, together with a box for every cell. A machine-readable element, such as a barcode, a QR code or the machine-readable zone (MRZ) of an identity document, has its format and its decoded payload. The payload of a barcode or QR code is never printed on the page, while the payload of an MRZ holds its printed lines. Text styles (i.e., bold, italic, underline, strikethrough, superscript, subscript) are annotated as character ranges.

The relations link elements across the document. A caption is linked to its figure, table or chart, and a footnote to its table. A paragraph, list item, or table split by a page break, or a paragraph that continues in the next column, is linked piece by piece with a *continues_in* relation, which also records whether a hyphen only splits a word (e.g., "per-" and "formance"). In contrast, a *stays_apart* relation marks two elements on consecutive pages that look continuous but must not be joined. Finally, a document may contain planted items, such as a misspelled word, a rare diacritic, or a near-miss name, which must be returned exactly as printed. The schema is detailed in Section A.

3.3 Annotation Procedure and Quality Assurance

We produce the ground truth of real documents with an internal annotation tool that uses the benchmark's own schema and validators, so that every saved file is a valid ground truth. The process comprises automatic drafting, annotator correction, and approval.

Automatic Drafting. Annotating whole documents by hand is slow and costly, so each page is first drafted automatically. The page is read by three vision-language models from different families (i.e., GPT-6.1 Sol, Claude Fable 5.1 and Gemini 4 Argon), and a value is accepted only when at least two of them read it alike, ignoring differences in case, spacing, and typographic marks (e.g., curly and straight quotes), while blind transcriptions are used to confirm it. Values without agreement are sent to a review queue, together with the region of the page from which they come. A judge model (GPT-5.6 Terra, with low reasoning effort) may suggest a value, but its advice alone never resolves a conflict. For PDF files, the native text layer is kept.

Annotator Correction. An annotator resolves every conflict in the queue and then corrects the draft in a visual editor, which covers boxes, classes, channels, text, reading order, relations, text styles, planted items, and table grids with merged cells. Chart data is not drafted and is entered by hand, whereas barcode payloads are decoded automatically and checked by the annotator.

Approval and Checks. Every save is validated against the schema and versioned, so that previous ground truth files can be recovered. A document is approved explicitly: the server refuses the approval while any conflict is still open, and any later edit withdraws it.

Table 1: Number of documents of the full corpus in which each track applies.

Track	Documents
T1 Text fidelity	459
T2 Reading order	415
T3 Structure and formatting	459
T4 Tables	329
T5 Machine-readable content	22
T6 Charts	54
T7 Visual grounding	481

The ground truth of synthetic documents is written by the generator together with the document, so it is exact by construction and it is checked automatically against the values stored in the file.

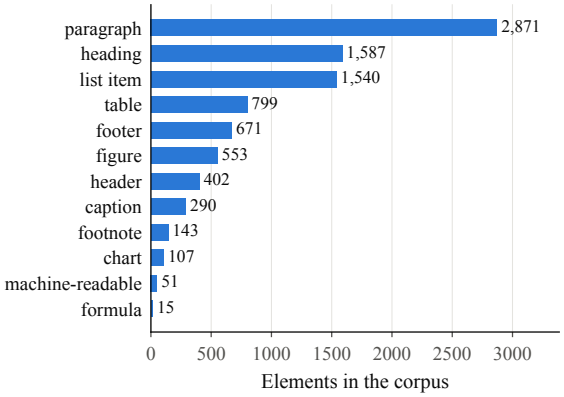


Figure 3: Number of ground truth elements of each class in the corpus.

3.4 Dataset Composition

WholeDoc Bench's full dataset contains 481 documents with 634 pages, of which 44 are synthetic and 437 come from the tagged PDF pool and the public collections. Of these, 10 documents form the public sample (Section 3.5) and the others the proprietary private set. Most documents are short runs of pages, since a run is cut only where no content continues across the break, with 430 documents of a single page and the longest one of 9 pages. The ground truth holds 9,029 elements, and Fig. 3 gives their number per class. Of these, 350 are born-digital PDF files, 119 are scanned PDF files and 12 are XLSX files. The documents are written in several languages, mainly English, French, Dutch and German. Table 1 gives the number of documents in which each track applies. Since a track applies only when the ground truth holds its content (Section 4.4), the tracks for charts and machine-readable content rest on fewer documents than the tracks for text and reading order, and their scores are less stable.

3.5 Public Sample and Private Set

A benchmark whose documents and ground truth are public can be optimized for, either by tuning a parser to its documents or by recognizing them when they are sent to a cloud service. We therefore split the corpus in two parts. Openbenchmarks [27] follows a similar practice: it releases its runner, scorer and a sample dataset, and keeps a held-out set that it does not share with its partners. The public sample contains 10 documents, selected to represent the cases that the benchmark evaluates, and is released with its ground truth, the adapters and the evaluation code. It shows the ground truth format and lets readers run the evaluation of Section 4 end to end on any parser, but it is too small to reproduce the scores of Section 5, which are computed on the full corpus. The other documents form the proprietary private set: they are scored with the same code, but which documents it contains and their ground truth are not released.

4 Evaluation Protocol

In this section, we describe what a parser must return, how its output is compared with the ground truth, and how the scores of the seven tracks are combined.

4.1 Parser Outputs and Adapters

Each parser is connected to the benchmark through an adapter, which calls the parser and converts its answer to one shared output format. A parser without an adapter can still be evaluated from its output files. For each document, the output has two parts. The first part is the Markdown of the whole document. The second part is an optional layout JSON, in which each item has a page, a class, a box and its text. An item may also give its position in the Markdown, the boxes of a table's cells, a chart's data and the font of a text block, together with the parser's own caption, footnote and continuation links. Boxes are converted to fractions of the page with one shared rule for every parser.

A parser that returns only Markdown is scored on every sub-metric that reads Markdown, and receives 0 on those that need the layout JSON. We keep these sub-metrics for two reasons. Firstly, the ground truth alone decides which sub-metrics apply (Section 4.4): if a missing output could remove a sub-metric, a parser could raise its mean by leaving out what it cannot do. Secondly, the box of an element is part of what downstream systems need, for example to show the place on the page that supports an answer. Since

every track score is reported separately, parsers that return only Markdown can still be compared on the tracks that read Markdown, which makes it easy to target specific and subjective use cases.

4.2 Normalization, Alignment, and Matching

Before every comparison, both texts are normalized in the same way: Unicode NFKC, straight quotes, a single kind of dash, Romanian comma-below letters, and single spaces. Diacritics are never removed and the inner separators of numbers are kept so that "1,234.56" and "1.234,56" are different words. Two measures are shared by several sub-metrics. The edit similarity of two texts is

$$\mathrm{sim}(a,b) = 1 - \frac{\mathrm{lev}(a,b)}{\max(|a|,|b|)}, \qquad \mathrm{sim}(\varepsilon,\varepsilon) = 1, \quad (1)$$

where lev is the Levenshtein distance over characters. The F1 score of an expected multiset E and a found multiset F counts each element at most as often as it is expected:

$$F_1(E,F) = \frac{2\,|E \cap F|}{|E| + |F|}, \qquad (2)$$

where $\cap$ takes, for every element, the smaller of its two counts. Most sub-metrics follow the same pattern: they pair the ground truth units (e.g., blocks, headings, chart points, or boxes) with the output units one to one and report the F1 score of the pairs.

The pieces of a *continues_in* chain are joined before any comparison. A hyphen marked as splitting a word is removed (e.g., "comp-" and "lex" give "complex"); any other hyphen at the break is accepted both with and without the hyphen, and every sub-metric uses the spelling that suits the output best. Finally, every output table is assigned to one track only. It is first linked to the layout item that holds it, by its position in the Markdown or by word overlap, and goes to T4 when that item is a table and to T6 when it is a chart. A table without such an item is linked to the ground truth table or chart with the highest word overlap.

4.3 Evaluation Tracks

Table 2 lists the 27 sub-metrics with their weights. Each score ranges from 0 to 1.

T1 Text Fidelity. T1 checks that the body text is complete and correctly spelled. The *bag-of-words F1* compares the words of the ground truth and of the output, in any order. The *sentence integrity* is the share of ground truth sentences that appear whole inside one output block. The *orthography* score is the share of planted items (e.g., typos, rare diacritics, near-miss names) that the output keeps exactly as printed, and the *block edit similarity* compares each ground truth block with its best output block, using Eq. (1), weighted by the length of the block.

Table 2: Sub-metrics of WholeDoc Bench, with their weights and the part of the prediction they read: Markdown (MD), layout JSON, either, or both. Tracks: T1 text, T2 reading order, T3 structure, T4 tables, T5 machine-readable content, T6 charts, T7 visual grounding.

	Sub-metric	Weight	Reads
T1	Word bag F1	1.0	MD
	Sentence integrity	1.0	MD
	Orthography	1.0	MD
	Block edit similarity	0.5	MD
T2	Order tests	1.0	MD
	Block order similarity	1.0	MD
T3	Block integrity	1.0	MD
	Heading tree	1.0	MD
	Relations	1.0	JSON
	Text styles	1.0	MD
	Page breaks	0.5	MD
	Typography	0.5	JSON
T4	TEDS	1.0	MD
	Header-matched rows	1.0	MD
	Table integrity	1.0	both
	Cell neighbor tests	0.5	MD
T5	Barcode payloads	1.0	MD
	MRZ identified	0.5	JSON
	MRZ text	1.0	JSON
	MRZ validity	0.2	JSON
T6	Data points	1.0	either
	Series	0.5	either
	Chart type	0.2	JSON
	Chart title	0.2	either
T7	Element boxes	1.0	JSON
	Element classes	0.5	JSON
	Table cell boxes	0.5	JSON

T2 Reading Order. From the ground truth, we generate order tests (a, b), in the spirit of the binary unit tests of olmOCR-Bench [30]: the last sentence of a unit, a, must come before the first sentence of the next unit, b. Let C_a and C_b be the positions of every copy of a and b in the output. A test passes only when both sentences are present and $\max C_a < \min C_b$, that is, every copy of a comes before every copy of b. The score is

$$\mathrm{OT} = \frac{\text{passed tests}}{\text{all tests}}. \qquad (3)$$

Hence, a parser that repeats text cannot pass on the copy that happens to be in the right place. This rule is stricter than the one of ParseBench [36], which compares the first copy of a with the last copy of b. A second sub-metric compares the order of all units with an edit distance.

T3 Structure and Formatting. The *block integrity* is the F1 of the ground truth paragraphs and list items returned as exactly one output block: a paragraph split into two blocks, or two paragraphs merged into one, does not match. The *heading tree* compares the headings found and their relative levels, the *relations* score is the F1 of the caption and footnote links, and the *text styles* score is the F1 of the styled characters, averaged over the styles. The *page breaks* score checks every marked page break: a continued paragraph must be one Markdown block, while a *stays_apart* pair must remain two. Finally, *typography* compares the font family, class and size of each block.

T4 Tables. We score each table with TEDS [38], in which a merged cell is one node with its span:

$$\mathrm{TEDS}(T, \hat{T}) = 1 - \frac{\mathrm{TED}(T, \hat{T})}{\max(|T|, |\hat{T}|)}, \tag{4}$$

where TED is the tree edit distance and $|T|$ the number of nodes. The *header-matched rows* score is the share of ground truth rows found under the right column labels, whatever the column order. The *table integrity* is the F1 of the ground truth tables returned as one table, even when they are split across pages, and joining two separate tables counts as a miss. The *cell neighbor tests* check that the rows after a page break stay under their headers.

T5 Machine-readable Content. The *barcode payloads* score is the share of barcode payloads found in the Markdown with their exact characters. Since a payload is never printed on the page, a parser can only pass by decoding the code. A machine-readable zone (MRZ) of an identity document is scored with three chained sub-metrics. A zone is *identified* when a layout item marked as an MRZ overlaps it with $\mathrm{IoU} \geq 0.5$. On an identified zone, the *MRZ text* score checks that the item's text equals its lines exactly, and the *MRZ validity* score checks that the parser's verdict on the check digits equals the one computed from the ground truth with the ICAO 9303 rules [12]. A zone that is not identified scores 0 on all three.

T6 Charts. The output points come from the chart data of the layout JSON and from the output tables linked to a chart (Section 4.2). In such a table, the row and column headers give the label and the series of each value cell, in either orientation, and every cell that reads as a number is one point. A data point with true value v matches an output point when its label and series match and

$$|\hat{v} - v| \leq \max(0.01\,|v|,\ 0.005\,(y_{\max} - y_{\min})), \tag{5}$$

where $[y_{\min}, y_{\max}]$ is the value axis of the chart. The *data points* and *series* scores are the F1 of the matched points and series names, the *chart type* is the share of charts with exactly the right type, and the *chart title* is the share of titles found.

T7 Visual Grounding. A ground truth element and a layout item on the same page are paired when their boxes overlap with $\mathrm{IoU} \geq 0.5$, as in COCO-style detection [16], and their words match with an F1 of at least 0.8 (charts, figures and machine-readable elements on the box alone). The *element boxes* score is the F1 of these pairs, the *element classes* score counts only the pairs of the same class, and the *table cell boxes* score is the F1 of the table cells located inside their own cell.

4.4 Score Aggregation and Coverage

Whether a sub-metric applies to a document is decided from the ground truth alone, so a parser cannot remove a sub-metric from the mean by leaving content out. A document whose parse failed (i.e., a missing or invalid prediction, or one with no text and no layout) receives 0 on every sub-metric that applies, and it stays in every mean.

Let $A_{d,k}$ be the sub-metrics of track k that apply to document d, with weights w_m (Table 2) and scores $s_{d,m}$. The score of track k on document d is the weighted mean

$$S_{d,k} = \frac{\sum_{m \in A_{d,k}} w_m\, s_{d,m}}{\sum_{m \in A_{d,k}} w_m}. \tag{6}$$

The score of track k on the corpus is the mean over the documents D_k in which the track applies,

$$S_k = \frac{1}{|D_k|} \sum_{d \in D_k} S_{d,k}, \tag{7}$$

and the overall score is the mean over the tracks K with a corpus score,

$$S = \frac{1}{|K|} \sum_{k \in K} S_k. \tag{8}$$

As in ParseBench [36], the weights apply inside a track, and the overall score is the unweighted mean of the track scores. Hence, a document adds one value to each track that applies to it, whatever its length, and every track counts equally in the overall score, whatever the number of documents it applies to.

Besides these scores, a coverage gate passes when at least 95% of the documents are parsed without failure. A parser that fails the gate keeps its scores, but is ranked below every parser that passes.

5 Evaluation of Document Parsers

In this section, we present the experimental setup for the benchmark and extended discussions on results and cost.

5.1 Experimental Setup

We evaluate 13 parsers in 19 configurations, grouped as follows.

Commercial Document APIs. Nutrient Data Extraction API in three modes (i.e., *structure*, *understand* and *agentic*), AWS Textract [1], Azure AI Document Intelligence [21], LlamaParse [18], Reducto [32], Extend [8] and Databricks `ai_parse_document` [5].

Vision-Language Models. Claude Fable 5.1 and GPT-6.1 Sol, each with medium and high reasoning effort. As in ParseBench [36], each page is sent as a separate request; from the second page on, the request also quotes the element that ended the previous page, so that the model can mark a paragraph, list item or table that continues (Section D).

Open-source Tools. Docling [7], PyMuPDF4LLM [4], pdftotext [34] and MarkItDown [20], run locally with their default settings; Docling also runs with its chart extraction, which is off by default (Section C).

Every parser is run on the full corpus, the public sample and the proprietary private set together, and scored by the same evaluation code. Each answer is converted to our output format by an adapter (Section 4.1), and the raw responses are kept, so that the scores can be recomputed without calling the parsers again.

5.2 Results and Discussion

Table 3 shows the score of every parser on each track, together with its overall score and its coverage. Nutrient in agentic mode has the best overall score (86.1), followed by its understand mode (80.8); the best vision-language model is Claude Fable 5.1 (67.1 at both efforts), and the best open-source tool is Docling (40.0). MarkItDown and pdftotext parse fewer than 95% of the documents and fail the coverage gate; a document in a format that a parser does not read counts as a failed parse.

The results show the following patterns:

1. **Text fidelity.** The cloud parsers keep most of the printed text (T1 from 77.4 to 94.6, except Nutrient structure at 71.9), and the vision-language models stay close to the best APIs (88.1 to 90.9). The tools that read only the text layer lose text on scanned pages (54.8 to 80.8).

2. **Reading order and structure.** Reading order is close to solved for the cloud parsers: Reducto (95.7), Databricks (94.9) and GPT-6.1 Sol (94.8) lead T2. Structure separates the parsers more: Nutrient understand and agentic (73.3 and 74.7) and the vision-language models (72.0 to 74.2) are close, while the other document APIs stay between 47.1 and 59.9.

3. **Tables.** LlamaParse Agentic Plus has the best T4 score (89.5), ahead of Nutrient agentic (87.1), Nutrient understand (86.1) and the vision-language models (82.4 to 84.9). AWS Textract (71.4) and Azure AI Document Intelligence (69.4) are lower, and MarkItDown (12.7) and pdftotext (0.0) return almost no usable tables.

4. **Machine-readable content.** Only three configurations decode barcodes: Nutrient in understand and agentic mode (100.0) and Azure AI Document Intelligence (94.3). Every other parser, the vision-language models included, scores at most 3.4.

5. **Charts.** Nutrient agentic recovers chart data best (64.5), followed by Claude Fable 5.1 (58.4 to 58.9), Databricks (51.0) and GPT-6.1 Sol (47.6 to 49.2), while Nutrient understand reaches 35.1. LlamaParse returns the data of a chart as an ordinary table, without marking it as a chart. Since an output table takes the track of the layout item that holds it (Section 4.2), these tables are scored in T4 and LlamaParse scores 0 in T6, as do Azure AI Document Intelligence and AWS Textract, which have no chart class. A parser that does not mark charts gives no way to tell a chart from a table in its output.

6. **Visual grounding.** Nutrient agentic (88.5) and understand (86.4) give the most accurate boxes, followed by Reducto (76.3), Extend (75.1 to 75.2) and Azure AI Document Intelligence (72.5). The vision-language models are lower (49.3 to 67.7), since they write their coordinates as text instead of measuring them. The open-source tools score 0 because their adapters return no layout (Section 4.1).

Discussion. The general-purpose models are close to the specialized parsers on text, reading order, structure and tables, which shows how far page reading by language models has come. Their weak points come from the way they produce output rather than from reading. Their prompt asks them to decode every barcode (Section D), yet they return the payload of almost none: a payload is a pattern of bars or modules that must be measured exactly, and a model that describes an image in text guesses it instead. Their bounding boxes are less accurate for a similar reason: they predict coordinates on a 0–1000 grid rather than measuring them on the page. The gap between Claude Fable 5.1 and GPT-6.1 Sol suggests that this ability varies across models, possibly because of differences in architecture and training data.

More expensive configurations improve some tracks, not all. LlamaParse Agentic Plus raises T4 by about 7 points over LlamaParse Agentic, but scores lower in T1 and T7. Agentic text in Extend changes every track by about one point or less. A higher reasoning effort changes the scores of both vision-language models by at most 2.1

Table 3: Results on the full corpus of WholeDoc Bench (%), sorted by overall score within each group. Overall is the unweighted mean of the track scores (Section 4.4), and Coverage is the share of documents parsed without failure. Parsers below 95% coverage are ranked after all others. Cost is in US dollars per 1,000 pages at list prices (Section 5.3); $\sim$ marks an estimate. The best score in each column is in bold and the second best is underlined. Tracks: T1 text fidelity, T2 reading order, T3 structure and formatting, T4 tables, T5 machine-readable content, T6 charts, T7 visual grounding.

Parser	T1	T2	T3	T4	T5	T6	T7	Overall	Coverage	Cost
Commercial document APIs										
Nutrient (agentic)	**94.6**	93.6	**74.7**	<u>87.1</u>	**100.0**	**64.5**	**88.5**	**86.1**	100.0	18.00
Nutrient (understand)	<u>92.7</u>	91.9	73.3	86.1	**100.0**	35.1	<u>86.4</u>	<u>80.8</u>	100.0	9.00
Azure AI DI	83.5	91.6	49.9	69.4	<u>94.3</u>	0.0	72.5	65.9	100.0	$\sim$27.60
Extend (Performance)	82.0	93.8	48.4	80.8	2.3	40.6	75.2	60.4	100.0	19.74
Databricks	85.7	<u>94.9</u>	47.1	77.4	2.3	51.0	64.1	60.4	97.5	$\sim$4.55
Extend (Perf. + agentic text)	81.9	94.0	48.3	80.2	2.3	39.6	75.1	60.2	100.0	20.59
Reducto (r-1)	81.0	**95.7**	51.1	76.4	0.0	38.5	76.3	59.9	100.0	$\sim$16.83
LlamaParse (Agentic Plus)	89.3	92.6	59.9	**89.5**	0.0	0.0	65.4	56.7	100.0	$\sim$56.25
LlamaParse (Agentic)	91.8	94.8	57.2	82.2	0.0	0.0	69.5	56.5	100.0	$\sim$12.50
AWS Textract	77.4	81.9	48.7	71.4	0.0	0.0	70.0	49.9	97.5	15.00
Nutrient (structure)	71.9	82.0	34.8	44.2	0.0	1.9	41.2	39.4	100.0	1.50
Vision-language models										
Claude Fable 5.1 (high)	90.9	93.4	72.2	84.7	3.4	<u>58.9</u>	66.0	67.1	97.5	179.30
Claude Fable 5.1 (medium)	90.9	93.2	72.0	84.9	2.3	58.4	67.7	67.1	97.5	169.87
GPT-6.1 Sol (medium)	89.2	94.8	<u>74.2</u>	82.8	2.3	47.6	49.3	62.9	97.5	$\sim$31.26
GPT-6.1 Sol (high)	88.1	94.0	72.2	82.4	2.3	49.2	49.4	62.5	97.5	$\sim$43.21
Open-source tools										
Docling	75.5	76.6	43.9	63.9	0.0	20.2	0.0	40.0	99.6	local
PyMuPDF4LLM	80.8	84.0	49.3	47.3	0.0	0.8	0.0	37.5	97.1	local
MarkItDown	54.8	64.8	15.9	12.7	0.0	1.3	0.0	21.4	83.4	local
pdftotext	56.6	68.9	13.4	0.0	0.0	0.0	0.0	19.8	80.0	local

points on every track, which suggests that parsing is limited by what the model perceives on the page rather than by how long it reasons about it. The Nutrient modes differ more: from structure to understand to agentic mode, the overall score rises from 39.4 to 80.8 and 86.1, so the cheapest mode is not a smaller version of the others, but represents a different trade-off, close to the text-layer tools in tables, charts and grounding.

Several results depend on what a parser returns at all, rather than on how well it reads. LlamaParse, AWS Textract and Azure AI Document Intelligence score 0 in T6 because they do not mark charts, and the parsers that do not decode barcodes lose one seventh of the overall score in T5. Reading order is almost saturated, with every cloud parser except two above 91 in T2, so harder multi-column and cross-page layouts are needed to separate the parsers on this track in future work. For a specific use case, the track scores are therefore a better guide than the overall score.

5.3 Score-Cost Correlation

Figure 4 compares the overall score of each cloud parser with its cost per 1,000 pages. The cost is computed from what each run consumed (i.e., tokens, pages or credits), as reported in the saved responses, and from the public pay-as-you-go list prices of each provider [2, 3, 6, 9, 10, 19, 22–26, 33], as checked on 2026-10-22 and 2026-10-23. Free tiers, volume discounts and negotiated prices are left out. When a response does not show the amount billed, the cost is an estimate. For the parsers that did not parse every document, the cost covers only the pages of the documents whose responses report their usage. The cost is reported beside the scores in Table 3 and never changes them or the ranking. The open-source tools run locally and have no price per page; therefore, they are not shown.

Four configurations form the best score-per-cost line: Nutrient structure ($1.50 per 1,000 pages), Databricks (60.4 at an estimated $4.55), Nutrient understand (80.8 at $9) and Nutrient agentic (86.1 at $18). Databricks reaches about the same overall score as Reducto and Extend at a quarter of their price, and LlamaParse Agentic keeps most of the text, reading order and table quality of LlamaParse Agentic Plus at less than a quarter of its cost ($12.50 against $56.25). The vision-language models are the most expensive: Claude Fable 5.1 costs about $170 to $179 per 1,000 pages, ten times Nutrient agentic, for an overall score 19.1 points lower. A higher reasoning effort does not pay off on this corpus: it raises the cost of Claude Fable 5.1 by 6% for a gain of 0.02 points, and the cost of GPT-6.1 Sol by 38% for a loss of 0.4 points. All costs are list prices; with volume discounts, negotiated prices or the estimates marked in Fig. 4, the order of configurations with close costs may change.

6 Conclusions and Future Work

We introduced WholeDoc Bench, a benchmark that evaluates document parsers on whole documents rather than single pages. Its ground truth links paragraphs, list items and tables split across page and column breaks, marks neighboring elements that look continuous but must stay separate, plants typos and near-miss names that must be returned as printed, and stores the decoded content of barcodes and machine-readable zones. One evaluation code scores seven tracks on the same documents, decides from the ground truth alone which sub-metrics apply and keeps failed documents in every mean. The evaluation code and a public sample of 10 documents are released, while the reported scores come from a proprietary private set, so that the evaluation can be run by anyone without exposing the scored documents to tuning.

The evaluation of 13 parsers in 19 configurations shows that no parser leads every track. Nutrient in agentic mode has the highest overall score, while Reducto leads reading order and LlamaParse Agentic Plus leads tables. General-purpose vision-language models come close to the specialized parsers on text, reading order, structure and tables, which shows how far these models have come. However, they decode almost no barcodes, give less accurate boxes, and are the most expensive configurations. Moreover, a higher reasoning effort does not improve their scores. Several results depend on what a parser returns at all rather than on how well it reads, such as a chart label or a decoded payload. For a specific use case, the track scores are therefore a better guide than the overall ranking.

The results have limits. The tracks apply to different numbers of documents (Table 1), so the tracks that rest on fewer documents, such as charts and machine-readable content, are less stable than the others. The open-source tools are scored without layout, since their adapters pass only Markdown (Section 4.1).

Future work will extend WholeDoc Bench in four directions. Firstly, the corpus will include other Office formats, such as DOCX, so that parsers are also evaluated on files that carry their structure in markup rather than on the page. Secondly, since reading order is almost saturated for the cloud parsers, harder multi-column and cross-page layouts are needed to separate them on this track. Thirdly, the adapters of some of the open-source tools that compute a layout, such as Docling, can pass it to the scorer, so that these tools are also compared on visual grounding. Finally, more documents with charts and machine-readable content will make the scores of these tracks more stable.

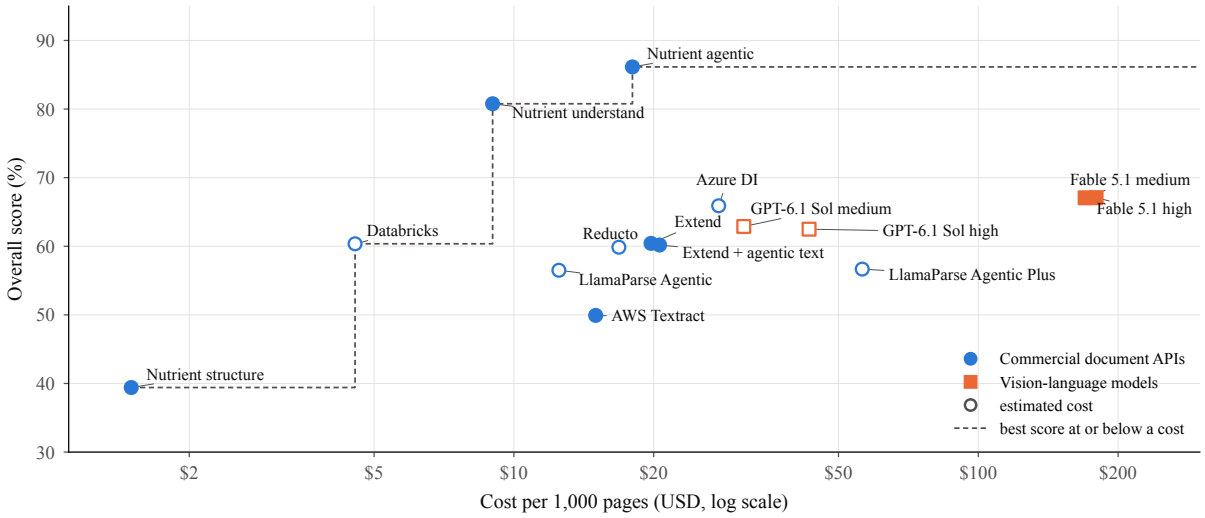


Figure 4: Overall score against cost per 1,000 pages (log scale), at public list prices on 2026-10-22 and 2026-10-23. Hollow markers are estimated costs. The dashed line is the best score reached at or below each cost. Local open-source tools are not shown.

References

[1] Amazon Web Services. Amazon textract developer guide. https://docs.aws.amazon.com/textract/latest/dg/what-is.html, 2026. Accessed: 2026-10-22.

[2] Amazon Web Services. Amazon textract pricing. https://aws.amazon.com/textract/pricing/, 2026. Accessed: 2026-10-22.

[3] Anthropic. Claude pricing. https://platform.claude.com/docs/en/about-claude/pricing, 2026. Accessed: 2026-10-22.

[4] Artifex Software. PyMuPDF4LLM. https://github.com/pymupdf/pymupdf4llm, 2026. Version 1.28.2. Accessed: 20 October 2026.

[5] Databricks. ai_parse_document function. https://docs.databricks.com/aws/en/sql/language-manual/functions/ai_parse_document, 2026. Accessed: 2026-10-26.

[6] Databricks. Ai functions pricing. https://www.databricks.com/product/pricing/ai-functions, 2026. Accessed: 2026-10-23.

[7] Docling Project. Docling. https://github.com/docling-project/docling, 2026. Version 2.131.0. Accessed: 20 October 2026.

[8] Extend. Extend documentation. https://docs.extend.ai, 2026. Accessed: 2026-10-22.

[9] Extend. How credits work. https://docs.extend.ai/general/how-credits-work, 2026. Accessed: 2026-10-22.

[10] Extend. Pricing. https://www.extend.ai/pricing, 2026. Accessed: 2026-10-22.

[11] Zhentao He, Can Zhang, Ziheng Wu, Zhenghao Chen, Yufei Zhan, Yifan Li, Zhao Zhang, Xian Wang, and Minghui Qiu. Seeing is believing? mitigating ocr hallucinations in multimodal large language models. *arXiv preprint arXiv:2506.20168*, 2025.

[12] International Civil Aviation Organization. *Doc 9303: Machine Readable Travel Documents*. ICAO, 8 edition, 2021. URL https://www.icao.int/publications/doc-series.

[13] Ameya Joshi, Joon Kim, Gus Eggert, Joseph Bajor, Cindy Hao, Jing Reyhan, Kushal Byatnal, and Eli Badgio. Realdocbench: A benchmark for field-level qa and layout understanding on real-world regulated documents. *arXiv preprint arXiv:2606.07401*, 2026.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[15] Zichao Li, Aizier Abulaiti, Yaojie Lu, Xuanang Chen, Jia Zheng, Hongyu Lin, Xianpei Han, Shanshan Jiang, Bin Dong, and Le Sun. Readoc: A unified benchmark for realistic document structured extraction. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 21889–21905, 2025.

[16] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014.

[17] Nikolaos Livathinos, Christoph Auer, Ahmed Nassar, Rafael Teixeira de Lima, Maksym Lysak, Brown Ebouky, Cesar Berrospi, Michele Dolfi, et al. Advanced layout analysis models for docling. *arXiv preprint arXiv:2509.11720*, 2025.

[18] LlamaIndex. LlamaParse. https://developers.llamaindex.ai/python/cloud/llamaparse/, 2026. Accessed: 2026-10-22.

[19] LlamaIndex. Llamaparse pricing. https://developers.llamaindex.ai/llamaparse/general/pricing/, 2026. Accessed: 2026-10-22.

[20] Microsoft. MarkItDown. https://github.com/microsoft/markitdown, 2026. Version 0.1.8. Accessed: 20 October 2026.

[21] Microsoft. Azure AI document intelligence: Layout model. https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/prebuilt/layout, 2026. Accessed: 2026-10-22.

[22] Microsoft. Document intelligence add-on capabilities. https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/concept/add-on-capabilities?view=doc-intel-4.0.0, 2026. Accessed: 2026-10-22.

[23] Microsoft. Azure ai document intelligence pricing. https://azure.microsoft.com/en-us/pricing/details/document-intelligence/, 2026. Accessed: 2026-10-22.

[24] Nutrient. Data extraction api pricing. https://www.nutrient.io/api/pricing/data-extraction-api/, 2026. Accessed: 2026-10-22.

[25] Nutrient. Dws data extraction pricing. https://www.nutrient.io/guides/dws-data-extraction/pricing/, 2026. Accessed: 2026-10-22.

[26] OpenAI. Api pricing. https://developers.openai.com/api/docs/pricing, 2026. Accessed: 2026-10-22.

[27] Openbenchmarks. Document processing benchmark. https://openbenchmarks.com/document-processing, 2026. Accessed: 2026-11-03.

[28] Linke Ouyang, Yuan Qu, Hongbin Zhou, Jiawei Zhu, Rui Zhang, Qunshu Lin, Bin Wang, Zhiyuan Zhao, Man Jiang, Xiaomeng Zhao, et al. Omnidocbench: Benchmarking diverse pdf document parsing with comprehensive annotations. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 24838–24848. IEEE, 2025.

[29] Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S Nassar, and Peter W J Staar. Doclaynet: A large human-annotated dataset for document-layout analysis. 2022. doi: 10.1145/3534678.353904. URL https://arxiv.org/abs/2206.01062.

[30] Jake Poznanski, Jon Borchardt, Jason Dunkelberger, Regan Huff, Daniel Lin, Aman Rangapur, Christopher Wilhelm, Kyle Lo, and Luca Soldaini. olmocr: Unlocking trillions of tokens in pdfs with vision language models, 2025. *URL https://arxiv.org/abs/2502.18443*.

[31] Siyi Qian, Xiongfei Bai, Bingtao Fu, Yichen Lu, Gaoyang Zhang, Xudong Yang, and Peng Zhang. Agentar-fin-ocr. *arXiv preprint arXiv:2603.11044*, 2026.

[32] Reducto. Reducto documentation. https://docs.reducto.ai, 2026. Accessed: 2026-10-22.

[33] Reducto. Credit usage. https://docs.reducto.ai/reference/credit-usage, 2026. Accessed: 2026-10-22.

[34] The Poppler Developers. pdftotext, Poppler PDF rendering library. https://poppler.freedesktop.org, 2025. Version 25.12.0. Accessed: 20 October 2026.

[35] Minglai Yang, Xinyan Velocity Yu, Pengyuan Li, Xinyu Guo, Zhenting Qi, Konwoo Kim, Longtian Ye, Xiaolong Luo, Jinhe Bi, Henry Zhang, et al. Dr. docbench: A comprehensive benchmark for expert-level and difficult document parsing. *arXiv preprint arXiv:2606.01393*, 2026.

[36] Boyang Zhang, Sebastián G Acosta, Preston Carlson, Sacha Bron, Pierre-Loïc Doulcet, Daniel B Ospina, and Simon Suo. Parsebench: A document

parsing benchmark for ai agents. *arXiv preprint arXiv:2604.08538*, 2026.

[37] Junyuan Zhang, Qintong Zhang, Bin Wang, Linke Ouyang, Zichen Wen, Ying Li, Ka-Ho Chow, Conghui He, and Wentao Zhang. Ocr hinders rag: Evaluating the cascading impact of ocr on retrieval-augmented generation. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 17443–17453. IEEE, 2025.

[38] Xu Zhong, Elaheh ShafieiBavani, and Antonio Jimeno Yepes. Image-based table recognition: data, model, and evaluation. *arXiv preprint arXiv:1911.10683*, 2019.

[39] Bangbang Zhou, Hangdi Xing, Yifan Chen, Jianjun Xu, Qi Zheng, Feiyu Gao, Zhibo Yang, Shuai Bai, Ming Yan, Jieping Ye, et al. Mpdocbench-parse: Benchmarking practical multi-page document parsing. *arXiv preprint arXiv:2605.22100*, 2026.

Appendix Contents

A Ground Truth Schema **15**
A.1 File Structure . . . 15
A.2 Elements . . . 15
A.3 Relations and Planted Items . . . 16
A.4 Validation Rules . . . 16
A.5 Example . . . 17

B Annotation Details **17**
B.1 Drafting and Review . . . 17
B.2 Selection and Sanitization . . . 18

C Parser Configurations **18**

D Prompts of the Evaluated Vision-Language Models **19**

E Additional Results **20**
E.1 Score against Cost per Track . . . 20

A Ground Truth Schema

In this appendix section, we give the complete ground truth format of WholeDoc Bench (schema version 0.1). Every ground truth file is validated when it is loaded: unknown fields are rejected, and a file that breaks any rule of Section A.4 is not accepted into the corpus.

A.1 File Structure

A ground truth file has the fields listed in Table 4. The reading order is stored once for the whole document, rather than on each element.

Table 4: Top-level fields of a ground truth file.

Field	Content	Required
`schema_version`	`"0.1"`	yes
`document`	id, SHA-256 of the file, capture (born-digital, scan or photo), origin of the ground truth, languages, and the size and printed label of every page	yes
`elements`	the blocks of every page (Section A.2)	yes
`relations`	links between elements (Section A.3)	no
`reading_order`	element ids in reading order, furniture excluded	no
`planted`	planted items, each with its kind and printed text	no

The origin of the ground truth tells whether the file was written by the generator together with a synthetic document, or drafted automatically and then corrected and approved by an annotator (Section B).

A.2 Elements

An element is one block on one page, and each piece of a split paragraph or table is an element of its own. Each element has an id, a page, a class, a channel (*body* by default), its printed text and a box (x, y, w, h) in fractions of the page size, with the origin at the top left. A box must lie on the page and have a non-zero area. Depending on its class, an element carries the additional fields of Table 5.

Table 5: Additional element fields.

Field	Content
`level`	level of a heading or list item, starting at 1
`header`, `rows`	table grid: header rows and body rows of cell texts
`spans`	merged cells, given by row, column and extent; the text sits in the top-left position and the covered positions are empty
`cells`	text and box of every table cell
`chart`	chart type (one of 34), series of named points with their values, value axis $[y_{\min}, y_{\max}]$, and title
`payload`, `format`	decoded content of a barcode or QR code, and its format; for a machine-readable zone, the format names the MRZ type (TD1, TD2, TD3, MRV-A, MRV-B) and the payload holds its lines; a barcode has no printed text
`styles`	bold, italic, underline, strikethrough, superscript and subscript as character ranges $[s, e)$ of the printed text; an empty list means that the text was checked and has no style
`font`	family, class (serif, sans, mono, script, decorative) and size in points

Table 6 lists the 13 classes and the five channels, together with the way the scorer treats each channel.

Table 6: Element classes and channels.

Classes	heading, paragraph, list item, caption, footnote, table, chart, figure, formula, code, header, footer, machine-readable; a page number is a header or a footer
body	scored in every track
furniture	running headers, footers and page numbers; left out of the body text, the headings and the reading order, but scored in T7
annotation	left out of the body text; neutral in T7
metadata	left out of the body text; neutral in T7
hidden	text present in the file but not printed; returning it costs nothing

A.3 Relations and Planted Items

A relation has a kind, a source element and a target element:

– *caption_of* links a caption to its figure, table or chart;

– *footnote_of* links a footnote to its table;

– *continues_in* links one piece of a paragraph, list item or table to the next piece, either in the next column or on the next page. Within a page, both pieces must be paragraphs or list items, and the link must go forward in the reading order. Each chain is a simple line without cycles, and the flag *hyphenated* marks a piece that ends in a hyphen splitting a word;

– *stays_apart* links two elements on consecutive pages that look continuous but must not be joined. Such a pair cannot also be linked by *continues_in*, and its two ends cannot belong to the same chain.

A planted item has a kind and its printed text. An *exact* item (e.g., a typo, a rare diacritic or a near-miss name) must be printed in the body text and must be returned exactly. *Hidden* and *redacted* items are text that the file contains but does not print.

A.4 Validation Rules

A ground truth file is rejected when it breaks any of the following rules:

– **Structure:** unknown fields; values outside a closed list (e.g., class, channel, style, chart type, relation kind); negative page numbers, levels below 1, or non-positive page and font sizes;

– **Ids and references:** duplicate element ids; an element on a page that does not exist; a relation or reading-order entry that refers to an unknown element;

– **Boxes:** a box outside the page, with zero area or with a non-finite value, for elements and table cells alike;

– **Text styles and fonts:** a style range that is empty, ends after the text, covers only white space or overlaps another range of the same style; a font without any field or with a blank family;

– **Tables:** cells or spans on an element that is not a table; a span over fewer than two cells, across header and body rows, outside the grid or over another span; a covered position that is not empty, or an empty position that no span covers; cell boxes that overlap by more than 1%;

– **Charts:** chart data on an element that is not a chart; a chart without a box or a type; duplicate series names or point labels; a series without points; a value axis with $y_{\max} \leq y_{\min}$; a blank title;

– **Machine-readable elements:** printed text on a machine-readable element; a blank payload; a payload on another class, or a payload that is printed on the page; a machine-readable zone without a box, or whose lines do not fit its format;

– **Relations:** the rules of Section A.3;

– **Corpus rules:** an *exact* planted item that is not printed in the body text; a hidden or redacted item shorter than four letters or digits, repeated, not contained in a hidden element, or printed on the page; a body heading without a level.

A.5 Example

The excerpt below shows part of a ground truth file: a paragraph that continues on the next page, with a hyphen that splits the word “per-formance”, and a footnote linked to its table.


```
{"schema_version": "0.1",
 "document": {"id": "report_two_column_pdf", "source": "born-digital",
  "groundtruth_source": "generator", "language": ["en"],
  "pages": [{"index": 0, "label": "1", "width": 595.276, "height": 841.89}, ...]},
 "elements": [
  {"id": "p5a", "class": "paragraph", "page": 0,
   "box": [0.5168, 0.3249, 0.3893, 0.0802],
   "text": "Training hours per person rose to 21. ... measured the per-"},
  {"id": "p5b", "class": "paragraph", "page": 1,
   "box": [0.0941, 0.0713, 0.5163, 0.016],
   "text": "formance of each course and dropped the two with the lowest scores."},
  {"id": "fn1", "class": "footnote", "page": 2,
   "box": [0.164, 0.1558, 0.3138, 0.0125],
   "text": "* Figures are unaudited and rounded to one decimal."}],
 "relations": [
  {"kind": "continues_in", "from": "p5a", "to": "p5b", "hyphenated": true},
  {"kind": "footnote_of", "from": "fn1", "to": "t1b"}]}
```


B Annotation Details

B.1 Drafting and Review

Figure 5 gives an overview of how a page is drafted, checked and corrected before its ground truth is approved.

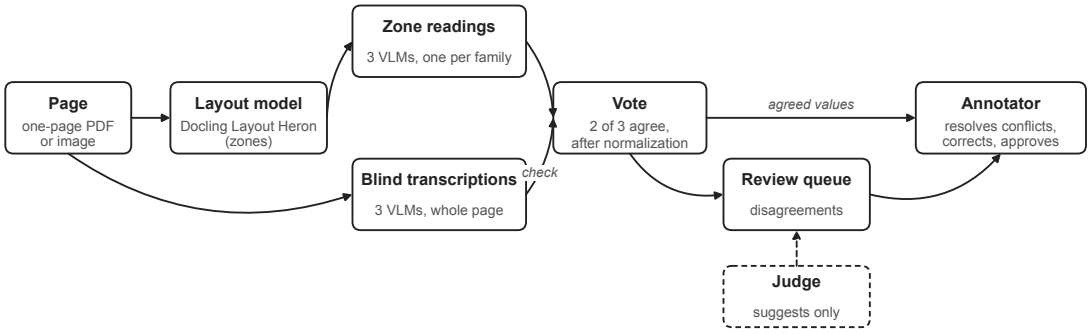


Figure 5: Drafting and correction of the ground truth of one page. Only values agreed by the panel, or decided by an annotator, enter the ground truth.

Input. Each page is drafted separately. A page of a PDF file is passed as a one-page PDF file, so that its native text layer is kept, and an image is passed at its own resolution. The page is first divided into zones by a layout detection model, Docling Layout Heron [17].

Readers. Each zone is then read by a panel of vision-language models from different families: GPT-6.1 Sol, Claude Fable 5.1 and Gemini 4 Argon. Every model runs with its provider’s default reasoning effort, and receives the cropped zones of the page. It returns, for each zone, its type, its text and, for a table, its cells. Charts are read in a second pass that returns their data as a table. In addition, every model transcribes the whole page *blind*, from the page image alone, without zones, without OCR and without the readings of the other models.

Agreement. Before comparing the readings, we normalize case, white space, quotes and dashes. A value is accepted when at least two of the three models read it alike; otherwise it goes to a review queue. A judge model (GPT-5.6 Terra, low reasoning effort) may suggest a value, but only the annotator resolves a conflict.

Review. For each conflict, the annotator sees the region of the page and the candidate readings, and accepts one of them or types the correct value (Fig. 6). The draft is then corrected in a visual editor, including its layout: the

annotator can draw, move, resize or delete boxes, change the class and channel of an element, and change the reading order. Relations and chart data are not drafted and are entered by hand: the table read from a chart is shown to the annotator as evidence only. Heading levels and barcode payloads are drafted, and the annotator can correct them. Every save is validated against the rules of Section A.4, and a document can be approved only when no conflict is open (Section 3.3).

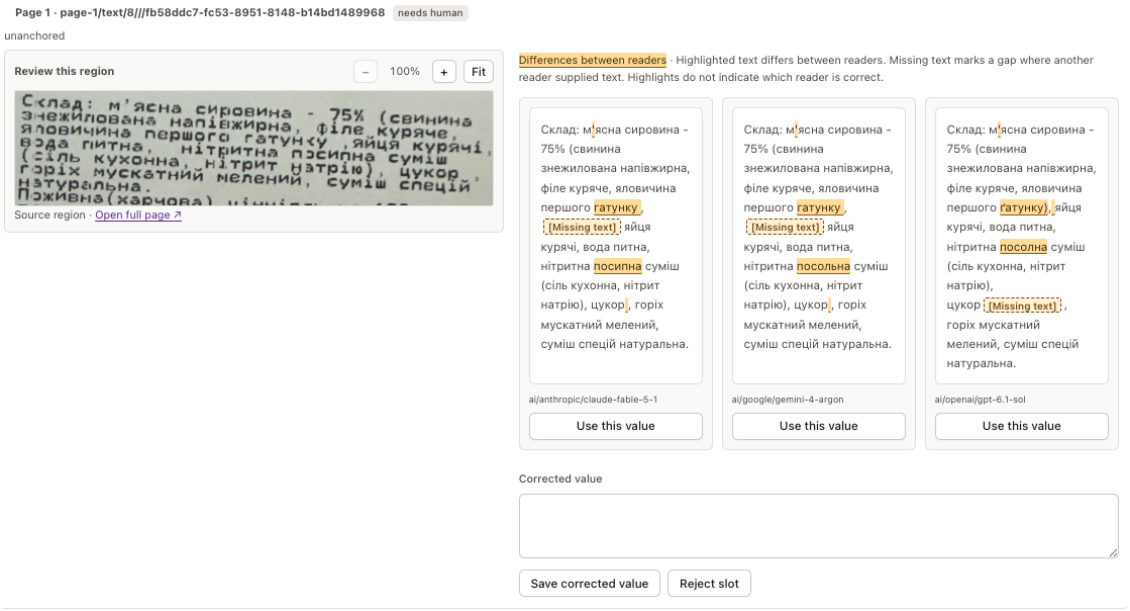


Figure 6: A conflict in the review queue.

B.2 Selection and Sanitization

Candidate documents were picked in batches for variety. Documents containing elements such as tables, charts and financial content are then carefully reviewed by hand. We rejected pages whose structure depends on a missing previous page, two-page spreads, and pages holding only a long uniform list. Every accepted document is sanitized: its pages are copied into a new PDF file, its form fields and annotations are flattened, and its metadata is removed.

C Parser Configurations

Table 7 lists the evaluated configurations. Every request pins the API or engine version when the service allows it, and turns off caching and result storage when the service offers it. The cloud services receive the whole document, except AWS Textract and the vision-language models, which receive one page per request, as one-page PDF files. A failed request is tried again up to three times, after a rate limit or a server error, within a time limit per document.

Table 7: Evaluated parser configurations. Layout: the adapter passes a layout JSON with element boxes to the scorer, besides the Markdown.

Parser	Configuration	Settings	Layout
Nutrient Data Extraction API	structure, understand, agentic	API 2026-05-25, latest engine	yes
AWS Textract	`AnalyzeDocument`	LAYOUT and TABLES features	yes
Azure AI DI	`prebuilt-layout`	API 2024-11-30; barcodes, formulas, high-resolution OCR and font styles on	yes
LlamaParse	Agentic, Agentic Plus	API v2, engine 2026-09-28; chart parsing at the same tier; continued tables merged	yes
Reducto	`r-1`	HTML tables, tables merged across pages, chart agent on figures	yes
Extend	Performance, Performance + agentic text	API 2026-02-09, engine 2.0.0; HTML tables, chart extraction on	yes
Databricks	`ai_parse_document`	REST API, schema 2.0, figure descriptions on	yes
Claude Fable 5.1	medium, high effort	`claude-fable-5-1`, at most 32,768 output tokens	yes
GPT-6.1 Sol	medium, high reasoning	`gpt-6.1-sol`, at most 32,768 output tokens	yes
Docling	default pipeline	chart extraction with Granite Vision on	no
PyMuPDF4LLM	default		no
MarkItDown	default	plugins off	no
pdftotext	plain mode	UTF-8	no

D Prompts of the Evaluated Vision-Language Models

Claude and GPT receive each page as a separate request, with the system and user prompts below. The prompts follow a similar guideline structure as the ones from ParseBench [36], extended with our schema. From the second page on, when the previous page ended with a paragraph, list item or table, the request also quotes that element, cut to its first 500 and last 1,500 characters when it is longer than 2,000. The model marks a continuation with `data-continues-previous`, which the adapter turns into a *continues_in* link.

System prompt (Claude Fable 5.1, GPT-6.1 Sol).

```
You are a document parser. Your task is to convert document pages to clean, well-structured
     markdown.

Guidelines:
- Preserve the document structure (headings, paragraphs, lists, tables)
- Convert tables to HTML format (<table>, <tr>, <th>, <td>)
- For existing tables in the document: use colspan and rowspan attributes to preserve
    merged cells and hierarchical headers
- For charts/graphs being converted to tables: use flat combined column headers (e.g., "
    Primary 2015" not separate rows) so each data cell's row contains all its labels
- For pictures, transcribe only the text printed in them; do not describe them
- Preserve any code blocks with appropriate syntax highlighting
- Maintain reading order (left-to-right, top-to-bottom for Western documents)
- Do not add commentary or explanations - only output the parsed content
- Transcribe text exactly as printed, with its spelling, punctuation, diacritics and
    numbers; do not correct or translate it
- Keep inline formatting as printed: **bold**, *italic*, <u>underline</u>, ~~strikethrough
    ~~, <sup>superscript</sup> and <sub>subscript</sub>
- Write formulas in LaTeX
- Leave out text that a reader cannot see on the page, such as text under a redaction box
    or text that is only in the PDF's text layer

Additionally, wrap each layout element in a <div> tag with:
- data-bbox="[x1, y1, x2, y2]" --bounding box in normalized 0-1000 coordinates where x is
    horizontal (left edge = 0, right edge = 1000) and y is vertical (top = 0, bottom =
    1000). x1,y1 is the top-left corner and x2,y2 is the bottom-right corner.
- data-label="<category>" --one of: Caption, Footnote, Formula, List-item, Page-footer,
    Page-header, Picture, Section-header, Table, Text, Title, Chart, Code, Barcode, MRZ
- data-id="<id>" --a short id, unique on the page, such as e1
```

```
Start a Title with # and a Section-header with ## to ###### by its level. Each List-item
    holds one item of a list. A page number is a Page-header or a Page-footer.
On a Caption, data-caption-of="<id>" names the Picture, Table or Chart it describes. On a
    Footnote that belongs to a table, data-footnote-of="<id>" names the table.
A Chart holds its data as an HTML table, with the chart's title as the table's <caption>,
    and data-chart-type="<type>" names its type, one of: bar, line, area, pie, scatter,
    bubble, histogram, box, violin, heatmap, treemap, funnel, radar, candlestick, step,
    stem, combo, density, errorbar, bullet, marimekko, pyramid, span, radial_bar, parallel,
     stream, donut, rose, sunburst, circle_packing, proportional_area, dot_matrix, gantt,
    waterfall.
A Barcode holds the decoded content of the barcode or QR code, or nothing when you cannot
    decode it. An MRZ is the machine readable zone of a passport, identity card or visa:
    write its lines in a code block, one per line, exactly as printed with every <, and set
     data-check-digits to "valid" when every check digit of the zone is right for what is
    printed, "invalid" otherwise.

On each Text, Title, Section-header, List-item, Caption, Footnote and Code element, name
    the font most of its text is printed in:
- data-font-family="<family>" --the typeface family, such as Arial, Times New Roman or
    Calibri, without weight or slope (Arial, not Arial Bold). Leave it out when you cannot
    tell the family.
- data-font-class="<class>" --one of: serif, sans, mono, script, decorative
- data-font-size="<points>" --the size in points at the page's printed size

When the user message quotes the element that ended the previous page, check whether this
    page's first element, after any Page-header or Page-footer, continues it: the same
    paragraph, list item or table carried over the page break. If it does, give that
    element the quoted element's data-label, add data-continues-previous="true" and
    transcribe only the part printed on this page. Leave the attribute out when the first
    element starts a new one, even when it looks like the quoted one.

Place elements in reading order. Every piece of content must be inside exactly one <div>
    wrapper.
```

User prompt, sent with every page.

```
Parse this document page and output its content as clean markdown, with each layout element
     wrapped in a <div data-bbox="[x1,y1,x2,y2]" data-label="Category"> tag. Use HTML
    tables for any tabular data. For charts/graphs, use flat combined column headers.
    Output ONLY the parsed content with div wrappers, no explanations.
```

Previous-page quote, added to the user message from the second page on.

```
The previous page ended with this element:

<div data-label="{label}">
{content}
</div>
```

E Additional Results

E.1 Score against Cost per Track

The overall score of Section 5.3 hides how the cost of a parser relates to each capability. Each figure below plots one track score of every cloud parser against its cost per 1,000 pages, at public list prices checked on 2026-10-22 and 2026-10-23, on a log scale. Hollow markers are estimated costs, and the dashed line is the best score reached at or below each cost. The axis of each figure covers the range of its scores. The open-source tools run locally and are not shown.

T1 Text fidelity. Text fidelity rises quickly with cost and then flattens: Databricks reaches 85.7 at an estimated $4.55, and every configuration above $9 scores between 77.4 and 94.6. Nutrient understand (92.7 at $9) and agentic (94.6 at $18) form the top of the line, and LlamaParse Agentic (91.8 at $12.50) is close to it. The vision-language models (88.1 to 90.9) cost two to twenty times more for a similar score.

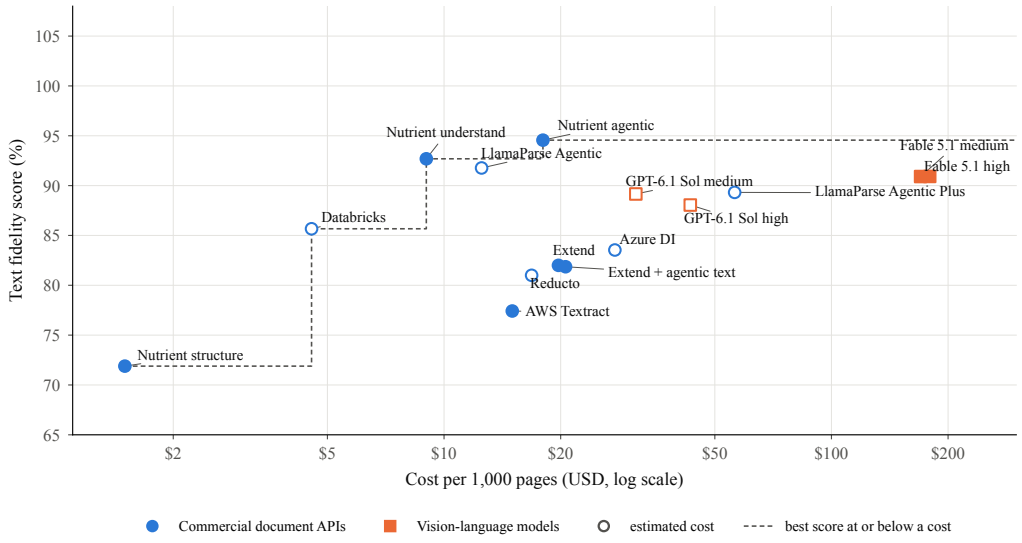


Figure 7: T1 text fidelity against cost.

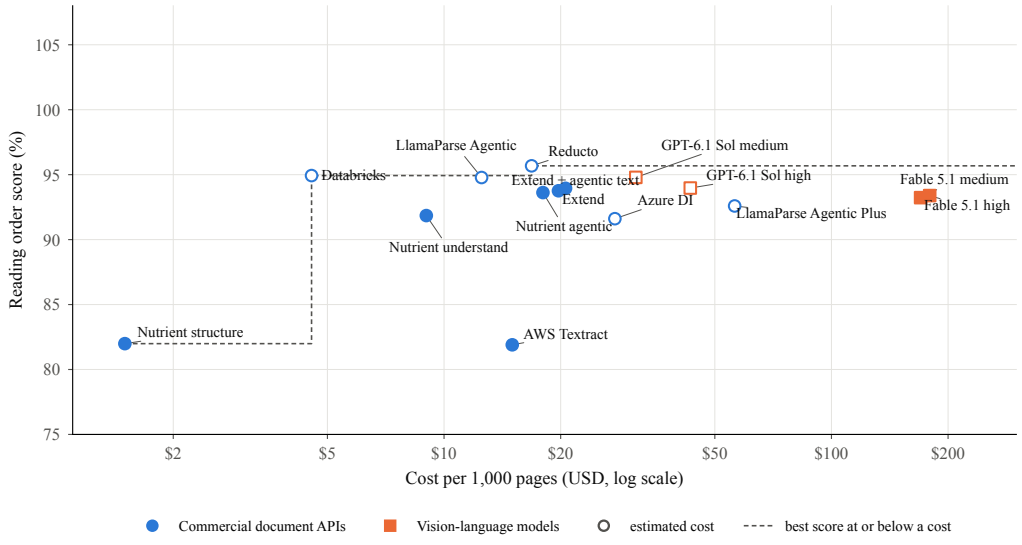


Figure 8: T2 reading order against cost.

T2 Reading order. Reading order is the track where cost matters least. Databricks already reaches 94.9 at an estimated $4.55, and only Reducto scores higher (95.7 at an estimated $16.83). All cloud parsers except AWS Textract and Nutrient structure lie between 91.6 and 95.7, so a higher price buys almost no improvement on this corpus.

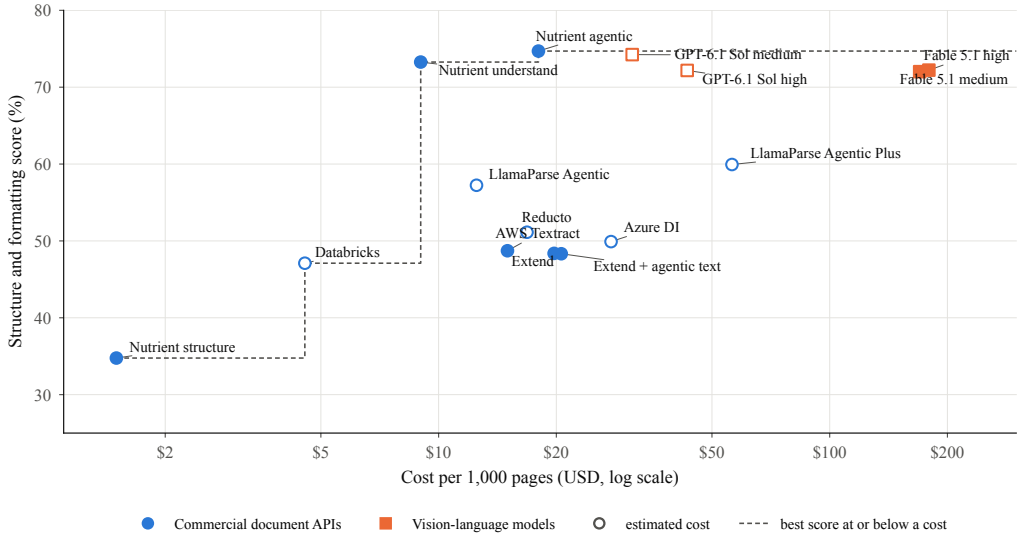


Figure 9: T3 structure and formatting against cost.

T3 Structure and formatting. Structure splits the parsers into two groups that do not follow cost. Nutrient understand and agentic (73.3 and 74.7) and the vision-language models (72.0 to 74.2) form the upper group, while the other document APIs stay between 47.1 and 59.9, whatever their price. Within the upper group, the vision-language models cost up to ten times more for the same score.

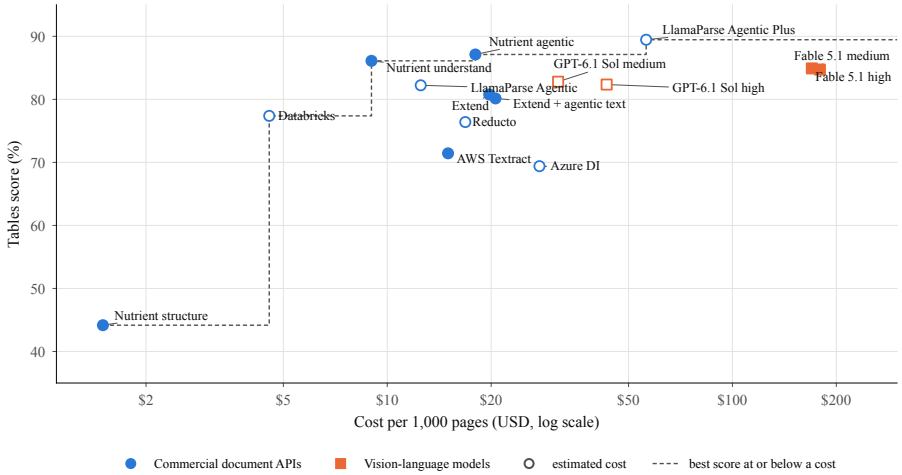


Figure 10: T4 tables against cost.

T4 Tables. Tables are the only track where the most expensive document API leads: LlamaParse Agentic Plus has the best score (89.5 at an estimated $56.25), 2.3 points above Nutrient agentic (87.1 at $18). Most cloud parsers lie between 76.4 and 86.1, and Databricks (77.4 at $4.55) gives most of that quality at the lowest price.

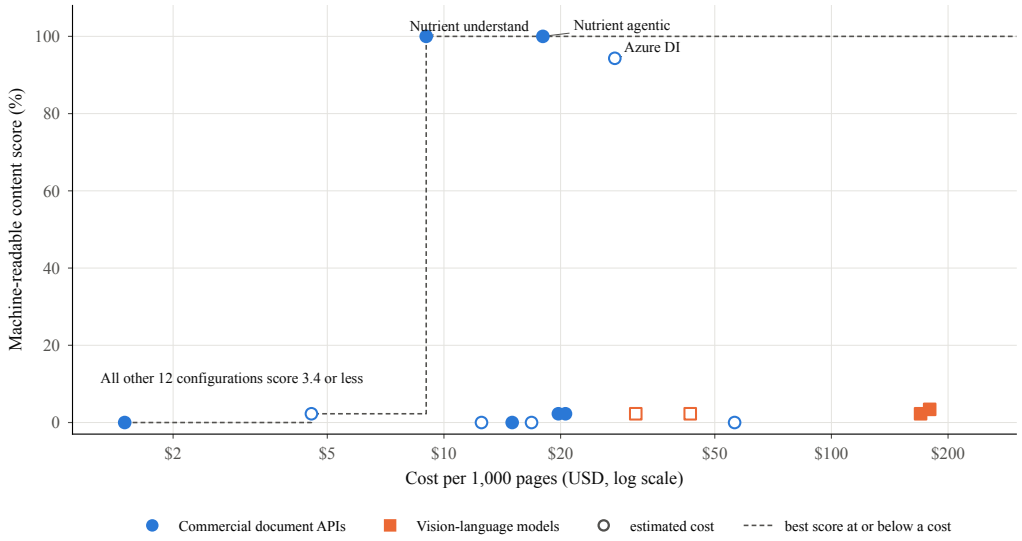


Figure 11: T5 machine-readable content against cost. Only three configurations decode barcodes; the others score 3.4 or less.

T5 Machine-readable content. Machine-readable content does not depend on cost at all: a parser either decodes barcodes or it does not. Nutrient understand and agentic (100.0) and Azure AI Document Intelligence (94.3) decode them, while the vision-language models, the most expensive configurations, score 2.3 to 3.4.

T6 Charts. Chart scores depend on whether a parser marks charts more than on its price. Among the parsers that mark them, Databricks (51.0 at an estimated $4.55) and Nutrient agentic (64.5 at $18) form the best line, and Claude Fable 5.1 (58.4 to 58.9) is the closest vision-language model at about ten times the cost. The parsers without a chart class score 0 at every price (Section 5.2).

T7 Visual grounding. Visual grounding separates the document APIs from the vision-language models. The APIs give their boxes from $1.50 up (41.2 to 88.5), with Nutrient understand and agentic (86.4 and 88.5) at the top and Reducto and Extend (75.1 to 76.3) behind them. The vision-language models score lower (49.3 to 67.7) at the highest prices, since they write their coordinates as text.

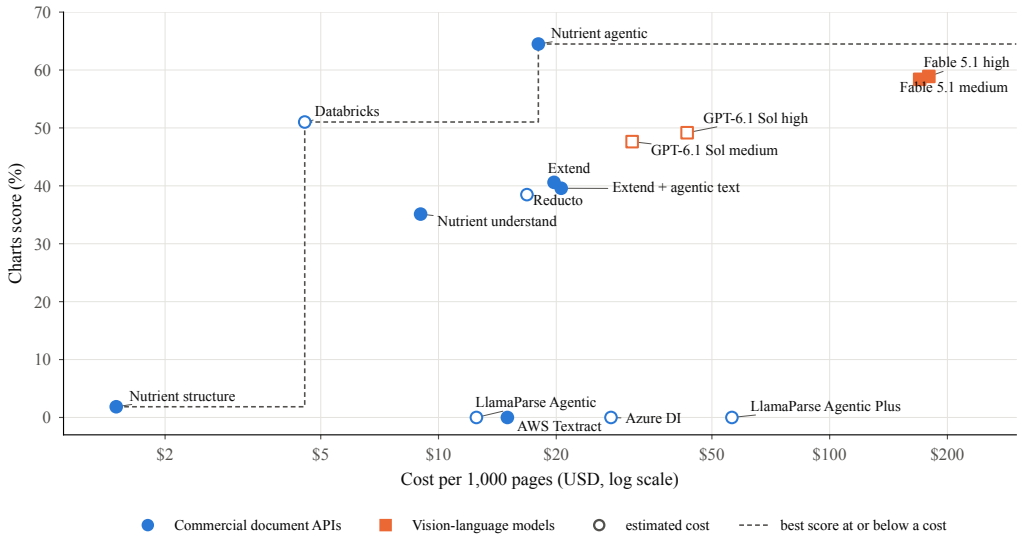


Figure 12: T6 charts against cost. Azure AI Document Intelligence, AWS Textract and LlamaParse mark no chart and score 0.

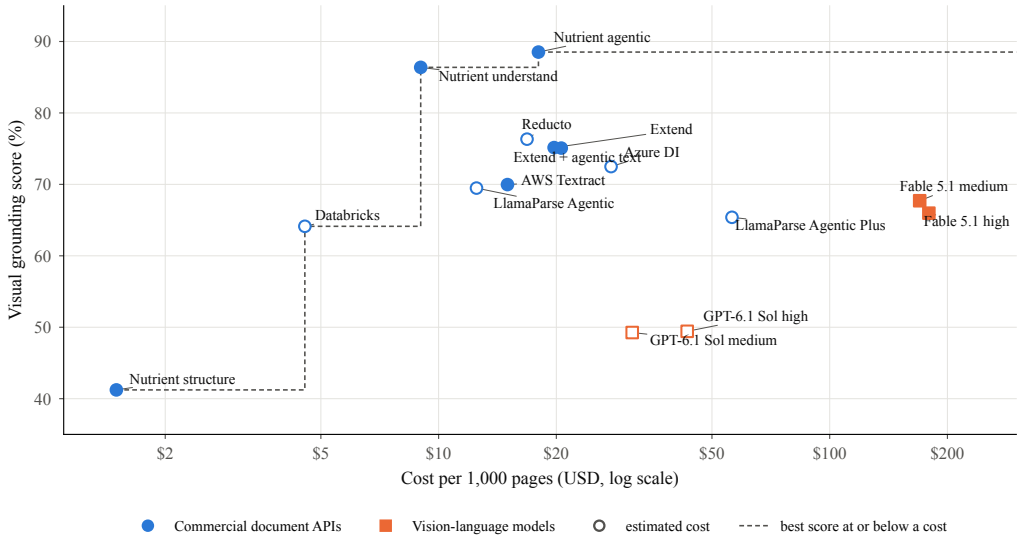


Figure 13: T7 visual grounding against cost.